

What BFI needs from Datadog, and why.

THE ASK, IN ONE SENTENCE

Teach our squads to **look at a trace before they add a log line, and teach our platform team to run the pipeline that makes that possible.**

WHY WE ARE TELLING YOU FIRST

We have spent six weeks measuring our own use of Datadog. We own more of it than we use, and the part we over-use is logging. A generic onboarding would miss that. This deck is what we found, so the workshops can start from where we actually are.

Who we are, in Datadog terms

A consumer-finance lender running a loan origination and servicing estate on GKE. Three languages, two shared wrapper libraries, one Datadog org, one two-year order.

~122 PRODUCTION SERVICES ON THE CLUSTER	71 OF THEM SEND LOGS OR TRACES TO DATADOG TODAY	3 LANGUAGES: GO (ABOUT TWO-THIRDS), JAVA SPRING BOOT, NODE.JS	2 WRAPPER LIBRARIES EVERY SERVICE IMPORTS – ONE GO, ONE JAVA
---	---	---	--

THE TWO WRAPPERS ARE THE LEVER

bfi-go-pkg gives every Go service its logger, HTTP middleware, gRPC interceptors and tracer setup. bravo-lib-logging (bfi-java-pkg) does the same for 18 Java services. **A default changed in one of them reaches the whole estate in one release.** Any workshop that ends with a wrapper change is worth more than thirty team sessions. We have two such changes in review now.

HOW WE KNOW WHAT FOLLOWS

Six weeks of review: 73 repositories read, production logs and spans queried in our org for every service, every production deployment manifest read, and the signed order Q-849776 (1 Oct 2025 – 30 Sep 2027) set against Datadog’s own estimated_usage metrics. **Every number in this deck comes from one of those four places.** Where a number needs your side to confirm it, we say so.

We are over on the two cheapest lines and under on the four expensive ones

Committed quantities per month from order Q-849776, against what `datadog.estimated_usage.*` reports for the 30 days to 14 September 2026.

LINE	COMMITTED / MONTH	USED, LAST 30 DAYS	READING
Infra hosts (Pro Plus)	100	59	under — 41% unused
APM Enterprise hosts	100	50	under — half unused
Indexed spans (15-day)	840M	100M	under — 12% of what was bought
Ingested spans	20,000 GB	12,900 GB	within
Containers	2,400	~3,095	over — small, at \$0.002 / container-hour
Log events indexed (3 + 7 day)	150M	~240M	over — 1.6×
Log ingestion	256 GB	~39,800 GB	over — about 150× , billed at \$0.10 / GB

Logs are 1.5% of what we bought and most of what we pay extra for. Teams log because nobody has shown them what a trace can answer. That is a training problem before it is a pricing one.

ONE NUMBER WE NEED YOU TO CONFIRM

40 TB across 240M events is 166 KB an event, which is not what a log line weighs. Either the estimate over-counts or a few services ship very large entries — we know of 64 KB request bodies being logged. **Please reconcile the ingestion metric against our invoices before either side uses it in a renewal conversation.**

The log stream is large, mostly unparsed, and mostly avoidable

4.5M PRODUCTION LOG LINES A DAY	93.6% OF THEM CARRY NO PARSED ATTRIBUTES — NO @LOGGER.NAME, NO LEVEL, NO TRACE ID	64% OF THE VOLUME COMES FROM FIVE SERVICES	16KB LINE SPLIT BY THE CONTAINER RUNTIME; EVERY LONGER LINE ARRIVES AS FRAGMENTS
---	---	--	--

WHERE IT COMES FROM

Request and response bodies logged at INFO. The Java wrapper defaults body logging to on; the Go wrapper is opt-in but eight production services have it on with an empty masking list. **Eight services run production at debug** . One service emits genuinely blank lines — 29% of everything. Serialised payloads, stack frames and the same error 20,000 times a day make up most of the rest. **Almost none of it is a log a human wanted.**

WHAT WE ARE DOING ABOUT IT OURSELVES

Forty-eight pull requests across the estate, two wrapper fixes, and a set of manifest changes ready for our SRE team. We hold ourselves to a gate before asking Datadog to ingest more: **under 2.5M lines a day, over 80% parsed.** The workshops on slide 10 are what get squads to keep it there — a team that can read a trace stops printing the payload.

The traces are there. Almost nobody looks at them.

The expensive half of the product is deployed and working. The squads who would benefit do not know it exists, and the features built on top of it are untouched.

42 PRODUCTION SERVICES COVERED BY APM	79M SPANS IN TWO DAYS, INCLUDING DATABASE AND QUEUE SPANS	3M+ SPANS A WEEK FROM THE WORKFLOW ENGINE ALONE – CONTROLLERS, JPA, OUTBOUND CALLS	12% OF THE COMMITTED INDEXED SPANS ACTUALLY USED
---	---	--	--

WHAT IS SWITCHED OFF, OR NEVER SWITCHED ON

Error Tracking — derived from telemetry already flowing, costs nothing extra, not used. **Database Monitoring** — the Cloud SQL Postgres instances are not enrolled. **Profiling** — not enabled anywhere. **Service Catalog** ownership — empty. When an incident happens, engineers open logs, because logs are where they were taught to look.

THE TRAINING GAP THIS DESCRIBES

Not “how to install the tracer” — it is installed. **How a service owner reads their own service page, finds the failed request, follows it across five services and a database call, and answers the question without a single log line.** Then how Error Tracking turns 20,000 identical errors a day into one issue with an owner.

Logs and traces are not joined up anywhere

0

PRODUCTION LOGS WITH A PARSED `@DD.TRACE_ID`, ACROSS TWO DAYS

21

SERVICES THAT EMIT `DD.TRACE_ID` AS PLAIN TEXT INSIDE THE MESSAGE — INJECTION IS ON, PARSING IS NOT

30 of 71

SERVICES SEND BOTH LOGS AND TRACES; THE REST SEND ONE OR THE OTHER

8

SERVICES CARRY A DIFFERENT `SERVICE` NAME IN LOGS THAN IN TRACES — DATADOG TREATS EACH AS TWO SERVICES

WHY

No log pipeline parses our JSON, so `dd.trace_id` stays a string in the message.

Unified service tagging is not applied on the pods, so names drift between the tracer and the log collector. And the **16 KB split** turns the JSON of any long line into fragments no parser can read. Nobody can click from a slow trace to its logs, or from an error log to the trace that caused it. We own the feature and get none of it.

WHAT WE NEED TO LEARN

Platform: pipelines and processors, trace-id remapping, reserved attributes, multi-line handling, unified service tagging on GKE. **Squads:** what their logger has to emit for the pipeline to work — MDC in Java, context loggers in Go — and how to check in the UI that it did.

Squads log bodies because they cannot see them — and the feature that would fix that is blocked

Every squad we asked gave the same reason for logging request and response payloads: “Datadog does not show them.” They are right, and Live Debugger is the answer. It does not work in our org today.

REMOTE CONFIGURATION IS FAILING

13 production services, about 91,000 failed polls a week, all with the same agent error: `empty targets meta in director local store`. Until it is fixed, no Live Debugger probe can be placed and no tracer setting can be changed from the UI. We have not found the cause on our side. **This is the single item where we most need Datadog’s help.**

THREE THINGS THAT LIMIT WHAT LIVE DEBUGGER CAN DO HERE

23 Dockerfiles download the tracer from `releases/latest` — unpinned, in production, so tracer behaviour changes without a deploy. **Three services run on OpenTelemetry** with dd-trace only indirectly, so they get no Datadog-native features at all. **Node.js gets line probes only**, and the Go estate — two-thirds of our services — is where dynamic instrumentation is thinnest.

If a squad can place a probe on a method and see the payload for one request, they will stop logging every payload for every request. That is the workshop with the largest effect on the ingestion line on slide 3.

Monitors that cannot fire, tests that test nothing, RUM half-configured

MONITORS

About 20 log monitors target `dev-`, `sit-` and `uat-` services. No non-production logs reach Datadog, so they report OK forever. The production monitors for one product line query a service name that does not exist. **38 monitors match the workflow engine; none watches whether a loan is progressing.** Alerts route to `@googlechat-test` and `@webhook-testwebhook`.

SYNTHETICS

Eight tests, all DNS and SSL checks. Nothing exercises whether a customer can submit an application. A five-week production defect on the front end went unreported by monitoring while 2.4M front-end errors accumulated in RUM.

RUM

Two browser applications instrumented. One has `allowedTracingUrls` and joins browser sessions to backend traces; the other does not, and its sample rates were left at the defaults. Session Replay is bought (600K sessions a month) and, as far as we can tell, never watched.

THE TRAINING GAP

Monitors as code — Terraform or API, reviewed like software, routed to channels people read. **Composite and workflow monitors** that watch a business step, not a container. **API synthetics** that exercise a real submission. **RUM setup** done once, correctly, on both apps.

Our own plan, so the workshops slot into it rather than duplicate it

PHASE 1 • NOW Platform fixes, no Datadog change Non-production log exclusion and retention, dead monitors fixed, tracer versions pinned, nine services taken off debug , masking lists set per service in the deployment manifests.	PHASE 2 • NEXT Squads cut log volume 48 pull requests open, one per repository, each naming the file and line. Two wrapper fixes: the Go scrubber masks numbers and lists; the Java Feign logger masks bodies and caps size.	PHASE 3 • GATED Ingestion and correlation Pipelines that parse our JSON, line reassembly, trace-id remapping, unified service tagging, per-service cost filters. This is where the platform workshops land.	PHASE 4 • ONE AT A TIME The paid features we already own Error Tracking, Live Debugger, Database Monitoring, real synthetics, RUM done properly. This is where the squad workshops land.
GATE 1 • PRODUCTION LOG LINES / DAY 4.5M → under 2.5M Protects the ingestion line. Squads own this.	GATE 2 • SHARE DATADOG CAN PARSE 6.4% → over 80% Cost filters, correlation and Error Tracking all work on parsed attributes. Platform owns this.	WHY THE ORDER MATTERS TO YOU If we enable ingestion features against today’s stream, Datadog faithfully reassembles the 64 KB bodies into single billable events and indexes them. The bill goes up and the trust goes down. We would rather arrive at the workshops with a stream half the size and ask you to help us use the features we bought.	

For developers: from “add a log line” to “open the trace”

Hands-on, in our own org, on our own services, in three language tracks. About eight squads; a cohort of two or three squads per session.

1	Traces first. A service owner opens their service page, finds the failed request from the last hour, follows it across five services and a database call, reads the span tags — and answers “what went wrong for this customer” without a log line. Then the same for “why is this slow”.	HALF DAY PER TRACK
2	Error Tracking as the inbox. How grouping works, how to assign an owner, how one issue replaces 20,000 identical log lines, how to alert on a new issue rather than on a log count.	2 HOURS ALL SQUADS
3	Logs Datadog can use. JSON, attributes versus message text, reserved attributes (<code>status</code> , <code>service</code> , <code>dd.trace_id</code>), what a 16 KB limit means, what never to log — bodies, identifiers, secrets — and how to check in the Log Explorer that your line parsed.	2 HOURS PER TRACK
4	Correlation from the application side. Log injection, unified service tagging, MDC in Spring, context loggers in Go, so the trace id lands as an attribute and not as text.	WITH #3
5	Live Debugger, once Remote Configuration works. Place a probe, capture one request’s payload, remove the probe. Java method probes; Node.js line probes; what Go can and cannot do today. This is the session that retires payload logging.	2 HOURS AFTER SLIDE 7

For the platform team: run the pipeline, own the cost, make the monitors real

1	Log pipelines and processors. JSON and grok parsers for our three logger formats, status and service remappers, trace-id remapping, multi-line and partial-line handling for the 16 KB split. Built together, on our stream, and left working.	1 DAY
2	Indexes, exclusion filters, retention and usage attribution. Which service costs what; per-index quotas and daily-quota alerts; when to exclude and when to archive; how to read the Usage & Cost page against the order — and settle the 40 TB question on slide 3.	HALF DAY
3	Agent and Remote Configuration on GKE. Why our 13 services fail to poll, how to pin and roll tracer versions, single-step instrumentation options, what the OpenTelemetry services give up.	HALF DAY + A SUPPORT CASE
4	Unified service tagging and the Service Catalog. Labels and annotations on the pod template so logs, traces and metrics agree on service , env , version ; ownership recorded once.	2 HOURS
5	Monitors as code, routing, SLOs. Terraform or API, reviewed like software; composite monitors on a business step; notification channels people actually read; SLOs on the loan workflow.	HALF DAY
6	Database Monitoring, RUM and synthetics. Enrol the Cloud SQL Postgres instances; allowedTracingUrls and sampling on both browser apps; one API synthetic that submits a real application to a test tenant.	HALF DAY

Eight weeks, in the order our own plan needs it

WEEKS 1-2 Platform first Pipelines, tagging, Remote Configuration, usage attribution (slide 11, items 1-4). These unblock everything else, and they are the ones only your engineers can teach on our org.	WEEKS 3-6 Squad cohorts “Traces first”, Error Tracking and structured logging (slide 10, items 1-4), three language tracks, while our Phase 2 cleanup lands. Each squad leaves having fixed one thing in its own service.	WEEKS 7-8 The features we own Live Debugger, Database Monitoring, monitors as code, RUM and synthetics — once the gates on slide 9 are met and Remote Configuration polls.
--	---	--

A	Reconcile the ingestion metric against our invoices. If 1.3 TB a day is right, it is the largest overage on the account and neither side should be surprised by it in October.	BEFORE WEEK 1
B	Open a case on Remote Configuration with us — 13 services, one error string, nothing we change on our side moves it.	WEEK 1
C	Tell us the roadmap for dynamic instrumentation in Go and for Node.js beyond line probes. Two-thirds of our services are Go.	WEEK 1
D	A named solutions engineer for the eight weeks, with time booked in our org, not slides.	THE WHOLE WINDOW

HOW WE WILL BOTH KNOW IT WORKED

Gate 1 and Gate 2 met and holding for four weeks. **Over 80% of production logs carry a parsed trace id.** **Every monitor can fire** and routes somewhere real. **Indexed-span usage rises** toward what was bought as log ingestion falls toward it. And in the next renewal conversation, the log lines and the span lines are sized from a year of data both sides trust.